

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación - 1PROG1

Programación II

Trabajo Práctico Integrador

Food Store – Sistema de Gestión de Pedidos

Integrantes:

Franco Sampieri

Mauro Di Pietro

Alan Gonzales

Felipe Videla

Tomas Martinez

Renzo Victoria

Índice

1. Marco Teórico.....	4
Java 21.....	4
Programación Orientada a Objetos (POO).....	4
Bases de Datos Relacionales.....	4
JDBC (Java Database Connectivity).....	4
Patrón DAO (Data Access Object).....	5
Arquitectura en Capas.....	5
2. Decisiones Técnicas y Arquitectura.....	6
Lenguaje y herramientas.....	6
Organización por paquetes.....	6
Conexión a la base de datos.....	6
Soft Delete.....	6
Transacciones en la creación de pedidos.....	6
3. Descripción de Capas y Paquetes.....	8
Capa de Entidades (Entity).....	8
Capa de Acceso a Datos (DAO).....	8
Capa de Servicios (Service).....	8
Capa de Presentación (Main).....	8
4. Entidades del Modelo (UML).....	9
Enumeraciones.....	9
Interfaz Calculable.....	9
5. Persistencia con JDBC.....	10
Configuración de la conexión.....	10
Uso de PreparedStatement.....	10
Try-with-resources.....	10
Mapeo de ResultSet a objetos.....	10
6. Menú de Consola.....	11
7. Reglas de Negocio Implementadas.....	12
8. Dificultades y Soluciones.....	13
9. Bibliografía / Webgrafía.....	14

1. Marco Teórico

Java

Java es un lenguaje de programación orientado a objetos, de propósito general y fuertemente tipado. Para este proyecto se utilizó Java 21, la versión LTS más reciente al momento del desarrollo. Java permite aplicar los cuatro pilares de la Programación Orientada a Objetos: encapsulamiento, herencia, polimorfismo y abstracción, los cuales son fundamentales para modelar correctamente las entidades del sistema.

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en torno a objetos, los cuales combinan datos (atributos) y comportamiento (métodos). Los conceptos aplicados en este proyecto son:

- Herencia: todas las entidades (Categoria, Producto, Usuario, Pedido, DetallePedido) heredan de una clase abstracta base llamada Entidad, que centraliza los atributos comunes id, eliminado y createdAt.
- Encapsulamiento: todos los atributos son privados y se accede a ellos mediante getters y setters.
- Polimorfismo e interfaces: se definió la interfaz Calculable con el método calcularTotal(), implementada tanto por Pedido como por DetallePedido, permitiendo que cada clase calcule su total de forma independiente.
- Clases abstractas: la clase Entidad actúa como clase base abstracta compartida por todas las entidades del dominio.

Bases de Datos Relacionales

Una base de datos relacional organiza la información en tablas relacionadas entre sí mediante claves primarias y foráneas. Para este proyecto se utilizó MySQL, un sistema de gestión de bases de datos relacional de código abierto ampliamente utilizado en la industria. El esquema incluye las tablas: categoria, producto, usuario, pedido y detalle_pedido.

JDBC

JDBC es la API estándar de Java para conectarse e interactuar con bases de datos relacionales. Permite ejecutar sentencias SQL desde código Java sin depender de un framework ORM. En este proyecto se utilizó JDBC puro, empleando las siguientes clases clave:

- Connection: representa la conexión activa con la base de datos.
- PreparedStatement: permite ejecutar sentencias SQL parametrizadas, evitando inyección SQL.

- **ResultSet:** permite recorrer los resultados de una consulta SELECT y mapearlos a objetos Java.
- **Try-with-resources:** garantiza el cierre automático de conexiones, statements y result sets.

Patrón DAO (Data Access Object)

El patrón DAO es un patrón de diseño que separa la lógica de acceso a datos del resto de la aplicación. Cada entidad tiene su propio DAO que contiene exclusivamente el código SQL y el mapeo de ResultSet a objetos. Esto facilita el mantenimiento, las pruebas y el reemplazo de la fuente de datos sin afectar la lógica de negocio.

Arquitectura en Capas

La arquitectura en capas organiza el código en niveles con responsabilidades bien definidas, donde cada capa solo se comunica con la inmediatamente inferior. Esto favorece la separación de responsabilidades, la legibilidad y el mantenimiento del código.

2. Decisiones Técnicas y Arquitectura

Lenguaje y herramientas

Se eligió Java 21 por ser el lenguaje requerido por la cátedra y por su solidez para aplicar POO. Como herramienta de gestión de dependencias se utilizó Maven, que facilita la incorporación del driver MySQL Connector/J (versión 9.0.0) sin necesidad de gestionar manualmente los archivos .jar.

Organización por paquetes

El proyecto se organizó en los siguientes paquetes, siguiendo la arquitectura en capas recomendada por la consigna:

Paquete	Responsabilidad
Config	Clase DatabaseConnection que centraliza la configuración de la conexión JDBC
Entity	Clases del modelo de dominio: Entidad, Categoria, Producto, Usuario, Pedido, DetallePedido, enums y la interfaz Calculable
DAO	Interfaces y clases de acceso a datos. Contiene todo el código SQL
Service	Lógica de negocio y validaciones. Coordina el uso de los DAOs
Main	Clase Main (punto de entrada) y MenuHandler (interacción con el usuario por consola)

Conexión a la base de datos

La configuración de la conexión está centralizada en la clase DatabaseConnection, que expone el método estático getConnection(). Esto evita repetir las credenciales en múltiples clases y facilita cambiar la URL, usuario o contraseña desde un único lugar.

Soft Delete

En lugar de eliminar físicamente los registros con DELETE, todas las bajas se implementaron como eliminación lógica: se actualiza el campo eliminado a TRUE en la base de datos. Esto preserva el historial y evita inconsistencias en tablas relacionadas, como detalles de pedidos que referencian productos eliminados.

Transacciones en la creación de pedidos

Al crear un pedido con múltiples detalles, cada operación de inserción se ejecuta de forma individual. Se consideró la implementación de transacciones JDBC (desactivando el autocommit y usando rollback en caso de error) para garantizar la consistencia de los datos. La versión implementada inserta primero el pedido y luego cada DetallePedido de forma secuencial.

3. Descripción de Capas y Paquetes

Capa de Entidades (Entity)

Contiene las clases del modelo de dominio puro. Ninguna clase de esta capa contiene SQL ni lógica de presentación. La clase abstracta Entidad define los atributos comunes:

- id (Long): identificador único generado por la base de datos.
- eliminado (boolean): campo para soft delete.
- createdAt (LocalDateTime): fecha y hora de creación del registro.

Capa de Acceso a Datos (DAO)

Cada entidad tiene su propio DAO. La interfaz genérica EntidadDAO<T> define los métodos comunes: selectAll(), selectById(), y eliminar(). Cada implementación agrega métodos específicos como insert(), update de campos individuales, y consultas adicionales (por ejemplo, selectByCategoria en ProductoDAO).

Capa de Servicios (Service)

Los servicios reciben datos desde el menú (Strings, números primitivos) y los convierten en objetos del dominio antes de pasarlos al DAO. También aplican validaciones de negocio, por ejemplo verificar que el precio no sea negativo, que el mail no esté vacío, o que el id ingresado sea válido.

Capa de Presentación (Main)

La clase MenuHandler contiene toda la lógica de interacción con el usuario: muestra el menú, lee la entrada con Scanner, y llama al servicio correspondiente. La clase Main únicamente instancia las dependencias e inicia el menú. Esta capa no contiene SQL ni lógica de negocio.

4. Entidades del Modelo (UML)

El diagrama UML del sistema define las siguientes clases y relaciones:

Entidad	Atributos propios	Relaciones
Entidad (abstracta)	id, eliminado, createdAt	Clase base de todas las entidades
Categoria	nombre, descripcion	1:N con Producto
Producto	nombre, precio, descripcion, stock, imagen, disponible	N:1 con Categoria
Usuario	nombre, apellido, mail, celular, contrasena, rol (Rol)	1:N con Pedido
Pedido	fecha, estado (Estado), total, formaPago (FormaPago)	Composicion 1:N con DetallePedido. Implementa Calculable
DetallePedido	cantidad, subtotal	N:1 con Producto. Implementa Calculable

Enumeraciones

- Rol: ADMIN, USUARIO
- Estado: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
- FormaPago: TARJETA, TRANSFERENCIA, EFECTIVO

Interfaz Calculable

Define el método `calcularTotal()` que es implementado por `Pedido` y `DetallePedido`. En `DetallePedido` calcula `cantidad * precio` del producto. En `Pedido` suma los subtotales de todos sus detalles.

5. Persistencia con JDBC

Configuración de la conexión

La clase `DatabaseConnection` centraliza la URL de conexión, el usuario y la contraseña. Expone el método estático `getConnection()` que devuelve un objeto `Connection` listo para usar. La URL configurada por defecto es `jdbc:mysql://localhost:3306/food_store`.

Uso de `PreparedStatement`

Todas las operaciones de escritura y lectura utilizan `PreparedStatement` en lugar de `Statement` plano. Esto previene inyección SQL al separar la estructura de la consulta de los datos que se pasan como parámetros.

Try-with-resources

Todas las conexiones, statements y result sets se abren dentro de bloques `try-with-resources`, lo que garantiza que se cierran correctamente al finalizar la operación, incluso si ocurre una excepción. Esto evita fugas de conexiones hacia la base de datos.

Mapeo de `ResultSet` a objetos

Cada DAO tiene métodos privados de mapeo (por ejemplo `mapCategoria`, `mapProducto`) que convierten una fila del `ResultSet` en un objeto Java. En el caso de `Producto`, el DAO realiza un JOIN con la tabla `categoria` para reconstruir el objeto `Categoria` anidado sin necesidad de hacer una segunda consulta.

6. Menú de Consola

El sistema presenta al iniciar el siguiente menú principal:

=== FOOD STORE ===

- 1. Categorías
- 2. Productos
- 3. Usuarios
- 4. Pedidos
- 0. Salir

Cada opción despliega un submenú con las operaciones CRUD disponibles: Listar, Crear, Editar y Eliminar. El menú valida todas las entradas del usuario: opciones fuera de rango, IDs inexistentes y errores de formato numérico son capturados y se muestra un mensaje claro antes de volver al menú.

Módulo	Operaciones disponibles
Categorías	Listar, Crear, Editar (nombre y/o descripción), Eliminar (baja lógica)
Productos	Listar, Crear, Editar (precio, stock y/o categoría), Eliminar (baja lógica)
Usuarios	Listar, Crear, Editar (nombre, apellido, mail, celular), Eliminar (baja lógica)
Pedidos	Listar, Crear con detalles, Editar (estado y/o forma de pago), Eliminar (baja lógica)

7. Reglas de Negocio Implementadas

Además del CRUD básico, el sistema respeta las siguientes reglas de negocio definidas en la consigna:

- Precio y stock no negativos: al crear o editar un producto, se valida que precio ≥ 0 y stock ≥ 0 . Si no se cumple, se informa el error y no se persiste el cambio.
- Pedido requiere usuario: no se puede crear un pedido sin seleccionar un usuario existente y no eliminado.
- Cantidad en DetallePedido mayor a cero: al agregar un detalle, se valida que la cantidad sea mayor a 0.
- Mail de usuario unico: el campo mail tiene restriccion UNIQUE en la base de datos. Si se intenta crear o editar un usuario con un mail ya existente, la excepcion SQL es capturada y se muestra un mensaje descriptivo.
- Soft delete en todas las entidades: ninguna operacion de eliminacion hace un DELETE fisico. Todas actualizan el campo eliminado = TRUE. Los registros eliminados no aparecen en listados ni pueden seleccionarse en operaciones.
- Categoria inexistente o eliminada: al crear un producto, se verifica que la categoria seleccionada exista y no este eliminada.

8. Algunas Dificultades y Soluciones

Dificultad	Solucion aplicada
NullPointerException al crear DetallePedido: el metodo setCantidad() llamaba a calcularTotal() antes de que el producto estuviera asignado.	Se corrigio el orden de los setters (primero setProducto, luego setCantidad) y se adopto el constructor con parametros DetallePedido(Producto, int) que inicializa ambos campos de forma segura.
Illegal operation on empty result set al insertar un DetallePedido: se llamaba a rs.next() sin verificar si el ResultSet tenia filas.	Se envolvio el ResultSet en un bloque try-with-resources y se agrego un if(rs.next()) antes de leer el ID generado. Lo mismo se aplico en los metodos insert() de los demas DAOs.
selectAll() en PedidoDAO traia datos incorrectos: estaba usando la query de detalles (con JOIN) en lugar de la query de pedidos.	Se corrigieron las referencias a las constantes SQL. SELECT_ALL_SQL para selectAll() y SELECT_ID_SQL para selectById().
mapProducto() fallaba al leer columnas del ResultSet: se usaban nombres con punto ("producto.id") que no existen en los alias definidos en el SQL.	Se corrigieron todos los nombres a su alias correcto con guion bajo: "producto_id", "categoria_id", etc., consistentes con los alias definidos en la query con AS.
selectDetallesPedidoById() traia los detalles de todos los pedidos en lugar del pedido especifico.	Se unificaron las constantes SQL para que SELECT_DETALLES_SQL ya incluya el WHERE dp.pedido_id = ?, eliminando la necesidad de una segunda constante y asegurando que el filtro siempre se aplique.
eliminarProducto() llamaba a categoriaService.eliminar() en lugar de productoService.eliminar().	Se corrigio la referencia al service correcto. El mismo error existia en eliminarProductosSinConsultar().

9. Bibliografia / Webgrafia

- Documentacion oficial de Java 21 – <https://docs.oracle.com/en/java/javase/21/>
- Documentacion de JDBC – <https://docs.oracle.com/javase/tutorial/jdbc/>
- MySQL Connector/J – <https://dev.mysql.com/doc/connector-j/en/>
- Maven – <https://maven.apache.org/>
- Patron DAO – <https://www.oracle.com/java/technologies/dataaccessobject.html>
- Consigna del TPI – UTN Tecnicatura Universitaria en Programacion a Distancia, Programacion 2, 2025